

# Resumo Analítico: Segurança Informática Moderna

---

*Baseado em Documentos da Biblioteca PS - SegInf*

## 1. Contexto Geral

### 1.1 Evolução da Segurança Informática

A segurança informática moderna migrou de soluções proprietárias para padrões abertos como OAuth 2.0, OpenID Connect, RBAC, PKI e TLS. Essa transição visa melhorar a interoperabilidade, segurança escalável e redução de custos no desenvolvimento e manutenção de sistemas.

Ameaças atuais incluem **phishing**, **roubo de tokens**, **ataques a Certificate Authorities (CA)**, **Cross-Site Scripting (XSS)**, **Cross-Site Request Forgery (CSRF)**, e **ataques de replay**.

### 1.2 Pilares da Segurança Moderna

- **Autenticação:** Verificar identidade do usuário (OAuth 2.0, OpenID Connect, Senhas)
- **Autorização:** Definir permissões de acesso (OAuth 2.0, RBAC, ACL)
- **Confidencialidade:** Proteger dados em trânsito (TLS 1.3, Cifras Simétricas)
- **Integridade:** Garantir que dados não foram alterados (Hash Functions, MAC)
- **Disponibilidade:** Garantir acesso contínuo (Redundância, Load Balancing)

## 2. Problema Central

### 2.1 Fragmentação Pré-Padrões

Antes do OAuth 2.0/OpenID Connect, cada aplicação gerenciava credenciais de forma isolada, resultando em soluções proprietárias não interoperáveis. Isso gerava alto custo de desenvolvimento, manutenção e risco de segurança.

### 2.2 Desafios Atuais

- **Equilíbrio entre segurança e usabilidade:** A busca por uma experiência do usuário simplificada (ex.: Single Sign-On - SSO) deve ser compatível com medidas de segurança rigorosas (ex.: PKCE, JWT, validação de tokens).
- **Complexidade de implementação:** Configurar provedores de identidade (IdP), gerenciar tokens e refresh tokens, e lidar com múltiplos fluxos de autenticação são tarefas complexas.
- **Ataques comuns:** XSS, CSRF, ataques de replay, e comprometimento de chaves privadas são ameaças constantes.

## 3. Principais Protocolos e Técnicas

### 3.1 OAuth 2.0

Framework de autorização delegada que permite que aplicações terceiras acessem recursos de usuários sem que estes compartilhem suas credenciais.

#### Participantes:

- **Resource Owner:** Usuário que possui o recurso.
- **Client:** Aplicação que solicita acesso.
- **Authorization Server:** Servidor que emite tokens.
- **Resource Server:** Servidor que hospeda o recurso protegido.

#### Fluxos Principais:

- **Authorization Code:** Código de autorização trocado por tokens (alta segurança, aplicações web/móveis)
- **Implicit:** Tokens retornados diretamente no URI (média segurança, aplicações legadas)
- **Client Credentials:** Credenciais do cliente para acesso não usuário (baixa segurança, serviços internos)
- **Resource Owner Password:** Credenciais do usuário trocadas por tokens (baixa segurança, aplicações de confiança)

#### Tokens:

- **Access Token:** Permite acesso a recursos.
- **Refresh Token:** Permite obter novos access tokens sem re-autenticação.
- **ID Token:** Contém informações do usuário (usado em OpenID Connect).

## 3.2 OpenID Connect (OIDC)

Protocolo de autenticação baseado em OAuth 2.0 que adiciona uma camada de autenticação, retornando informações do perfil do usuário em um **ID Token (JWT)**.

### Componentes:

- **ID Token (JWT):** Contém claims sobre o usuário.
- **UserInfo Endpoint:** Retorna informações adicionais do usuário.
- **Nonce:** Valor aleatório para prevenir ataques de replay.

### Fluxo Authorization Code + PKCE:

1. Cliente redireciona usuário para IdP com `response_type=code` e `scope=openid`.
2. IdP autentica usuário e retorna código de autorização.
3. Cliente troca código por ID Token e Access Token.
4. Cliente valida JWT e obtém informações via UserInfo Endpoint.

## 3.3 RBAC (Role-Based Access Control)

Controlo de acessos baseado em papéis que permite definir permissões com base em papéis atribuídos aos usuários.

### Vantagens:

- Escalabilidade.
- Facilidade de gestão.

### Desvantagens:

- Role explosion (muitos papéis).
- Dificuldade em modelar permissões complexas.

### Modelos RBAC:

- RBAC0: Papéis básicos.
- RBAC1: Hierarquia de papéis.
- RBAC2: Restrições (ex.: separação de funções).
- RBAC3: RBAC1 + RBAC2.

### 3.4 PKI (Public Key Infrastructure)

Infraestrutura de chaves públicas que permite estabelecer confiança em comunicações digitais.

#### Componentes:

- **Certificados X.509:** Contêm chave pública e informações do titular.
- **Certificate Authorities (CA):** Emitem e assinam certificados.
- **CRL (Certificate Revocation List):** Lista de certificados revogados.
- **OCSP (Online Certificate Status Protocol):** Verificação em tempo real.

#### Validação de Cadeia de Confiança:

1. Verificar assinatura do certificado.
2. Verificar validade (data de expiração).
3. Verificar revogação (CRL/OCSP).
4. Verificar emissor confiável.

### 3.5 TLS 1.3

Protocolo para comunicação segura que garante confidencialidade e integridade em comunicações na web.

#### Principais melhorias sobre TLS 1.2:

- Remoção de algoritmos inseguros (RC4, SHA-1).
- Forward Secrecy obrigatória.
- Handshake mais rápido (1 RTT).

### 3.6 JCA (Java Cryptography Architecture)

Arquitetura para serviços criptográficos em Java que permite flexibilidade e interoperabilidade.

## 4. Conceitos Técnicos Simplificados

### 4.1 JSON Web Token (JWT)

Formato de token assinado ou encriptado usado para transmitir informações de forma segura entre partes.

**Estrutura:** HEADER.PAYLOAD.SIGNATURE

**Exemplo de Payload:**

```
{
  "iss": "https://accounts.google.com",
  "sub": "1234567890",
  "aud": "123-abc.apps.googleusercontent.com",
  "exp": 1516239022,
  "iat": 1516239022,
  "name": "Jorge Silva",
  "email": "jorge@example.com"
}
```

## 5. Metodologias

### 5.1 JCA em Java

**Exemplo de uso:**

```
// Geração de chave AES
KeyGenerator keyGen = KeyGenerator.getInstance("AES");
SecretKey secretKey = keyGen.generateKey();

// Encriptação
Cipher cipher = Cipher.getInstance("AES/CBC/PKCS5Padding");
cipher.init(Cipher.ENCRYPT_MODE, secretKey);
byte[] encrypted = cipher.doFinal("texto".getBytes());
```

## 6. Conclusões e Implicações Práticas

### 6.1 Conclusões dos Autores

- OAuth 2.0 e OpenID Connect são padrões de facto para autenticação/autorização.
- RBAC é escalável mas requer gestão cuidadosa de papéis.
- PKI é fundamental para confiança em comunicações.
- TLS 1.3 é obrigatório para comunicações seguras.
- JCA permite flexibilidade e interoperabilidade em Java.

### 6.2 Implicações Práticas

#### Benefícios:

- Redução de custos com gestão de identidade.
- Melhor experiência do usuário (SSO).
- Segurança aprimorada com PKCE e JWT.
- Interoperabilidade entre plataformas.

#### Desafios:

- Complexidade inicial na configuração.
- Dependência de terceiros (IdPs).
- Vulnerabilidades se mal implementado.
- Limitações de escopo em alguns fluxos.

### 6.3 Recomendações

- Usar fluxos seguros: Authorization Code Flow com PKCE para aplicações web/móveis.
- Validar tokens: Sempre verificar assinaturas, `iss`, `aud`, `exp`.
- Armazenar tokens: HttpOnly cookies ou armazenamento seguro no dispositivo.
- Monitorizar atividades suspeitas: Tentativas de login em múltiplos dispositivos.

## 7. Termos-Chave

- OAuth 2.0
- OpenID Connect (OIDC)
- JSON Web Token (JWT)
- ID Token
- Access Token
- Refresh Token
- PKCE (Proof Key for Code Exchange)
- RBAC (Role-Based Access Control)
- PKI (Public Key Infrastructure)
- TLS 1.3
- JCA (Java Cryptography Architecture)
- Scopes
- Nonce
- Certificate Authority (CA)
- X.509 Certificate
- Forward Secrecy
- Cipher Suite
- Hash Function
- Symmetric Encryption
- Asymmetric Encryption

## 8. Ambiguidades e Sugestões

- **Falta de dados numéricos:** Nenhum documento apresenta métricas de adoção ou comparações quantitativas entre fluxos.
- **Casos de uso específicos:** Pouca informação sobre implementação em IoT ou sistemas legados.
- **Recomendação:** Consultar RFCs oficiais ([RFC 6749](#), [RFC 7519](#), [RFC 7662](#)) e documentação de provedores.



## 9. Referências

- [RFC 6749 - OAuth 2.0](#)
- [RFC 7519 - JWT](#)
- [RFC 7662 - OAuth 2.0 Token Introspection](#)
- Documentos da biblioteca PS - SegInf